INSTITUTO SUPERIOR TECNOLÓGICO PRIVADO IDAT CARRERA DE DESARROLLO DE SISTEMAS FRONT-END Y BACK-END - EEST VIRTUAL

CURSO

Desarrollo de Servicios Web

TEMA

Parcial definitivo

DOCENTE

Torres Segovia Eduar Enrique

INTEGRANTES

Jimenez Rojas, Jonathan Jose

Revilla Huapaya, Javier Alberto

Talledo Ceverino, Andy Steve

Lima - Perú

2026

API de Gestión de Citas Médicas

1. Introducción

El presente proyecto consiste en el desarrollo de una API REST para la gestión de citas médicas.
La aplicación permite registrar, consultar, actualizar y eliminar información relacionada con
pacientes, médicos y citas médicas.

La API fue desarrollada utilizando ASP.NET Core y el lenguaje C#. El proyecto fue realizado
con fines académicos para demostrar el funcionamiento de endpoints HTTP, controladores,
interfaces y lógica de aplicación.

La información se almacena temporalmente en memoria dentro de la aplicación, por lo que no
se utiliza una base de datos externa.

2. Arquitectura del Sistema

La arquitectura del sistema se basa en una API REST. Los clientes pueden interactuar con
la aplicación mediante solicitudes HTTP.

Estructura general:

```
Cliente (Postman / Aplicación)
        |
        v
API REST (ASP.NET Core)
        |
        v
```

Controladores

|

v

Almacenamiento en memoria

Durante el desarrollo se utilizó Postman para realizar pruebas de los endpoints.

3. Componentes del Sistema

Controladores:

Reciben las solicitudes HTTP y ejecutan la lógica necesaria para cada operación.

Modelos:

Representan las entidades principales del sistema como Paciente, Médico y Cita.

Interfaces:

Se utilizan para definir comportamientos comunes, como el sistema de notificaciones.

Almacenamiento en memoria:

Los datos se almacenan en colecciones dentro de la aplicación mientras esta se encuentra en ejecución.

4. Endpoints de la API - Pacientes

GET /api/pacientes

Obtiene la lista de pacientes registrados.

GET /api/pacientes/{id}

Obtiene un paciente específico por su identificador.

POST /api/pacientes

Registra un nuevo paciente.

PUT /api/pacientes/{id}

Actualiza la información de un paciente existente.

DELETE /api/pacientes/{id}

Elimina un paciente del sistema.

5. Endpoints de la API - Médicos

GET /api/medicos

Obtiene la lista de médicos registrados.

GET /api/medicos/{id}

Obtiene un médico específico.

POST /api/medicos

Registra un nuevo médico.

PUT /api/medicos/{id}

Actualiza la información de un médico.

DELETE /api/medicos/{id}

Elimina un médico del sistema.

6. Endpoints de la API - Citas

GET /api/citas

Obtiene todas las citas registradas.

GET /api/citas/{id}

Obtiene una cita específica.

POST /api/citas

Registra una nueva cita médica.

PUT /api/citas/{id}

Actualiza una cita existente.

DELETE /api/citas/{id}

Elimina una cita.

7. Flujo de Registro de Citas

1) El cliente envía una solicitud POST a la API.
2) El controlador recibe la solicitud.
3) Se validan los datos recibidos.
4) Se registra la cita en la colección en memoria.
5) Se ejecuta el sistema de notificaciones.
6) La API devuelve una respuesta al cliente.

8. Cálculo de Edad del Paciente

En el sistema no se muestra directamente la fecha de nacimiento del paciente al consultar la información. En su lugar se calcula la edad automáticamente.

La edad se obtiene restando el año actual menos el año de nacimiento del paciente y verificando si ya cumplió años en el año actual.

```
[Required(ErrorMessage ="La fecha de nacimiento es OBLIGATORIO")]
9 referencias
public DateTime FechaNacimiento { get; set; }

[Required(ErrorMessage ="El correo electrónico es OBLIGATORIO")]
[EmailAddress(ErrorMessage = "El formato del correo no es válido")]
9 referencias
public string Correo { get; set; } = "";

2 referencias
public int Edad
{
    get
    {
        int edad = DateTime.Now.Year - FechaNacimiento.Year;
        // Condicionamos a que si aún no llega su fecha de nacimiento
        // del año en curso, le reste 1 año a su edad
        if (DateTime.Now.DayOfYear < FechaNacimiento.DayOfYear)
        {
            edad--;
        }

        return edad;
    }
}
```

9. Estado de los Médicos

Cada cita posee un estado que puede ser:

Activo

Inactivo

Esto permite controlar si un médico se encuentra se encuentra en actividad o no sin necesidad de eliminar completamente la información.

10.Sistema de Notificaciones

El sistema implementa un mecanismo de notificaciones basado en una interfaz.

Las notificaciones se ejecutan cuando se realizan operaciones importantes como:

- Registro de una cita
- Actualización de una cita
- Eliminación de una cita

11.Interfaz de Notificación

La interfaz INotificacion define el comportamiento que deben implementar los diferentes tipos de notificación.

Estructura conceptual:

```
INotificacion
   |-- NotificacionSMS
   |-- NotificacionCorreo
   |-- NotificacionWhatsApp
```

Este diseño permite que el sistema sea flexible y extensible.

12.Validaciones del Sistema

La API incluye validaciones para asegurar que los datos enviados por el cliente sean correctos.

Entre las validaciones implementadas se encuentran:

- Campos obligatorios (Required / StringLength)
- Validación de datos nulos
- Validación de formatos de entrada

Estas validaciones permiten evitar el registro de información incompleta o incorrecta.

13.Pruebas de los Endpoints

Para validar el correcto funcionamiento de la API se realizaron pruebas utilizando Postman.

Las pruebas incluyen:

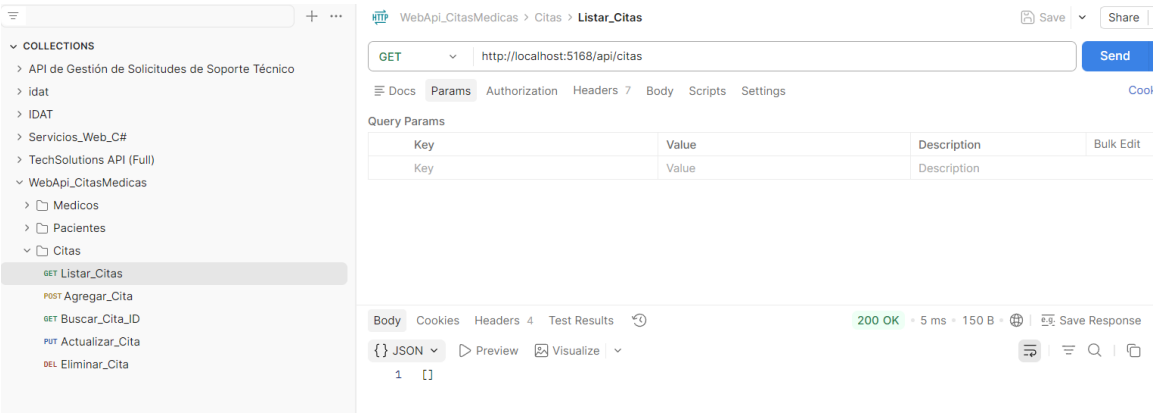
- CRUD citas
- CRUD pacientes
- CRUD médicos
- Notificaciones en consola VSC (citas)
- Validaciones

En esta sección se agregarán capturas de pantalla que muestran:

- Ejecución de cada endpoint.
- Respuestas del servidor.
- Validaciones de campos obligatorios.
- Manejo de errores cuando faltan datos requeridos.

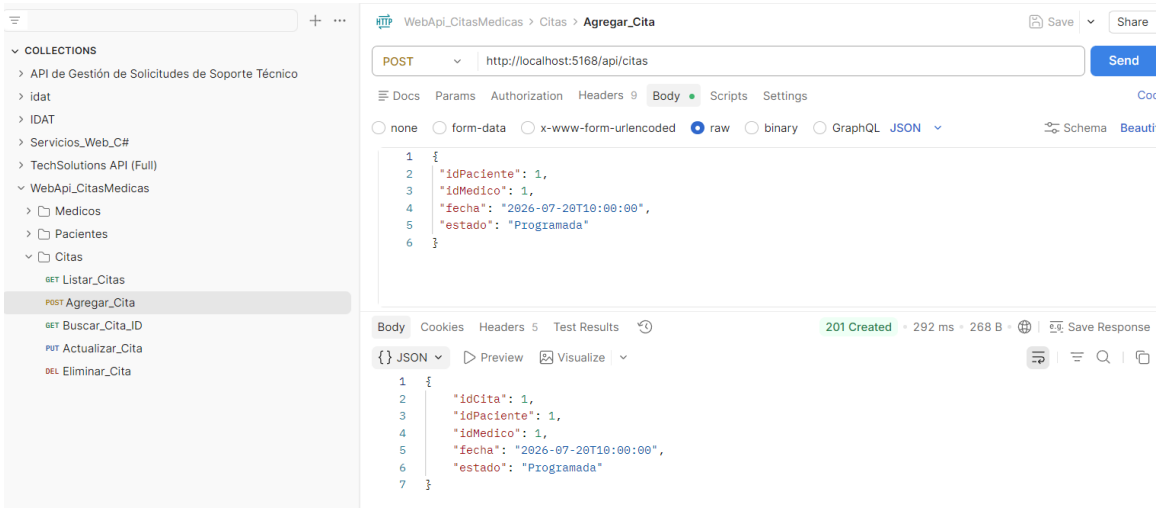
13.1. Prueba GET Cita /api/citas

Listar_Citas: En esta prueba se verifica que el endpoint devuelve correctamente la lista de citas registradas en el sistema (vacía).

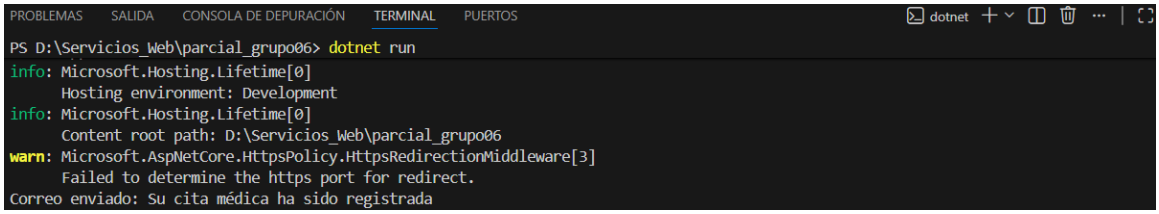


13.2. Prueba POST registro de Cita /api/citas

Agregar_Cita: En esta prueba se verifica que el endpoint devuelve correctamente 201 Created, agregando la cita la lista de citas.

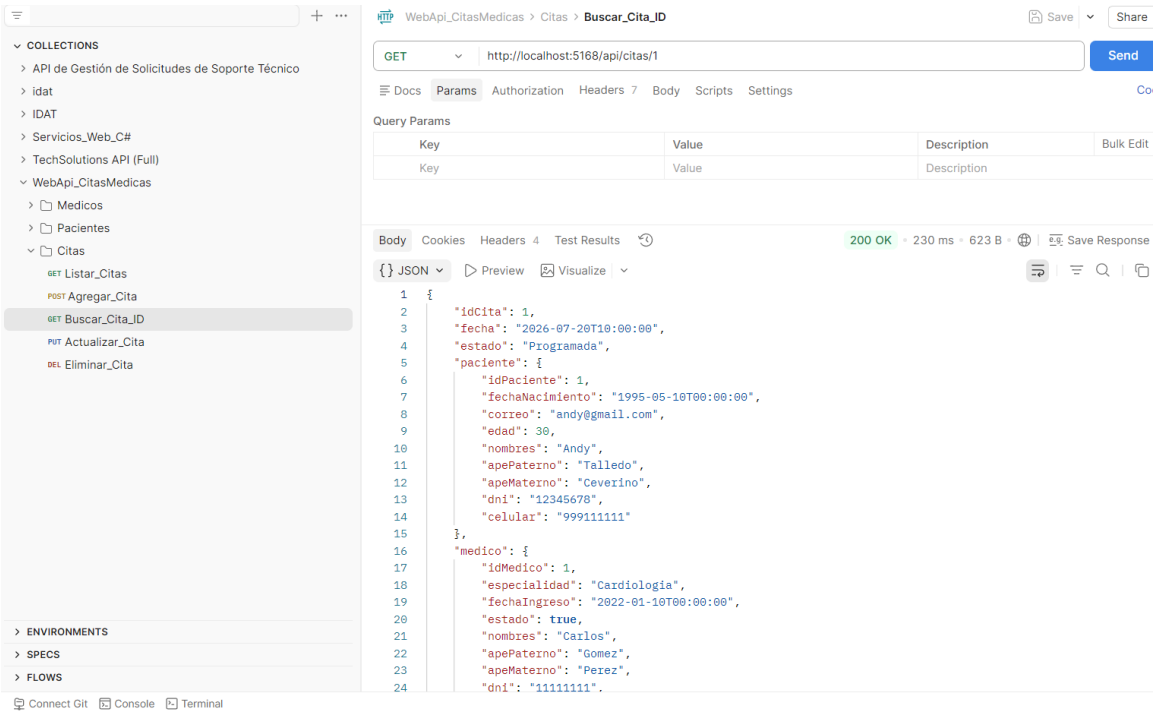


- Notificación en consola por **correo** de la cita registrada.

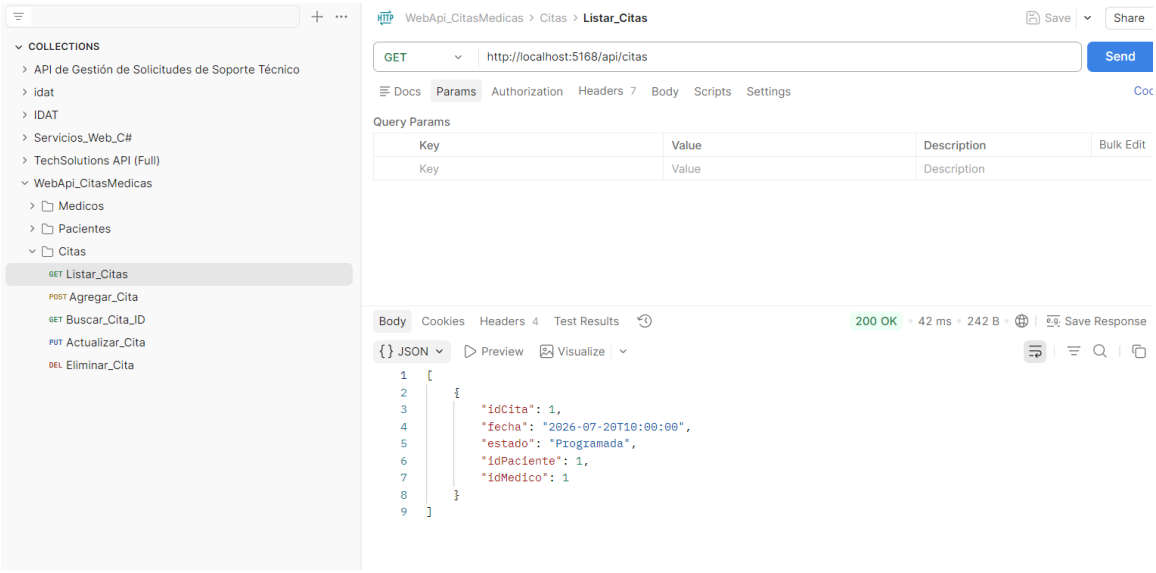


13.3. Prueba GET búsqueda por ID Citas /api/citas/1

Buscar_Cita_ID: En esta prueba se verifica que el endpoint devuelve correctamente la cita buscada de acuerdo con su ID.

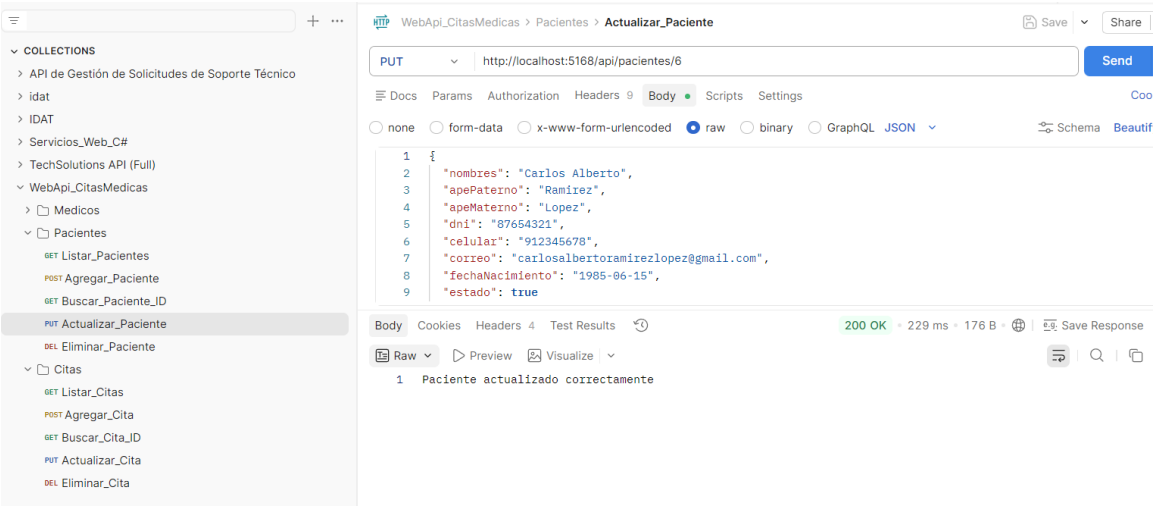


- Si buscamos por GET (lista de la colección completa), mostrará campos IdPaciente e IdMedico con sus respectivos códigos, pudiendo obtener el detallado del paciente y médico de la cita solo al buscar por ID la cita (imagen anterior adjunta).

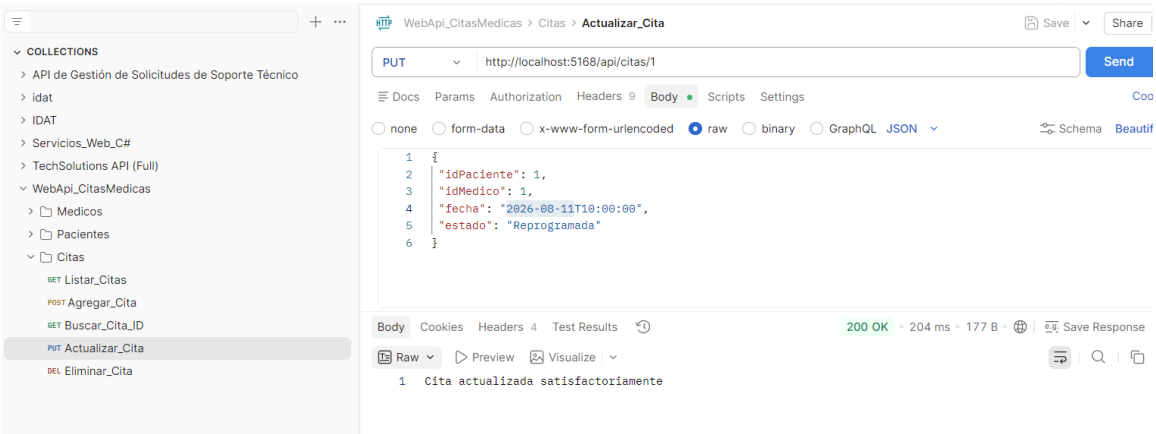


13.4. Prueba PUT actualizar Cita /api/citas/1

Actualizar_Cita: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, actualizando la información de la cita con ID 1.



- Fecha cambió de 2026-07-20 a 2026-08-11.
- Estado cambió de “Programada” a “Reprogramada”.

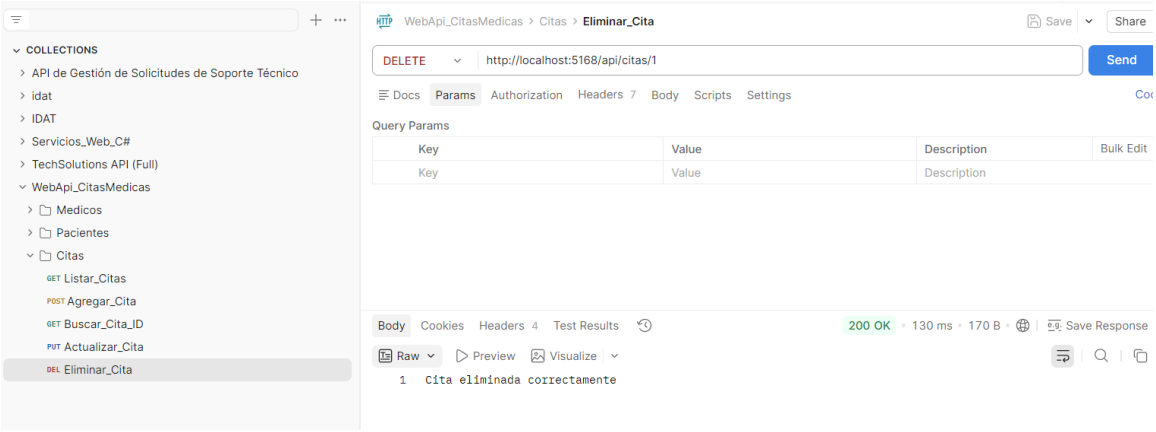


- Notificación en consola por **SMS** de la modificación cita.

```
PROBLEMAS    SALIDA    CONSOLA DE DEPURACIÓN    TERMINAL    PUERTOS                dotnet
PS D:\Servicios_Web\parcial_grupo06> dotnet run
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\Servicios_Web\parcial_grupo06
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]
      Failed to determine the https port for redirect.
Correo enviado: Su cita médica ha sido registrada
SMS enviado: Su cita médica fue actualizada
```

13.5. Prueba DELETE eliminar Cita /api/citas/1

Eliminar_CIta: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, eliminando la cita con ID 6.

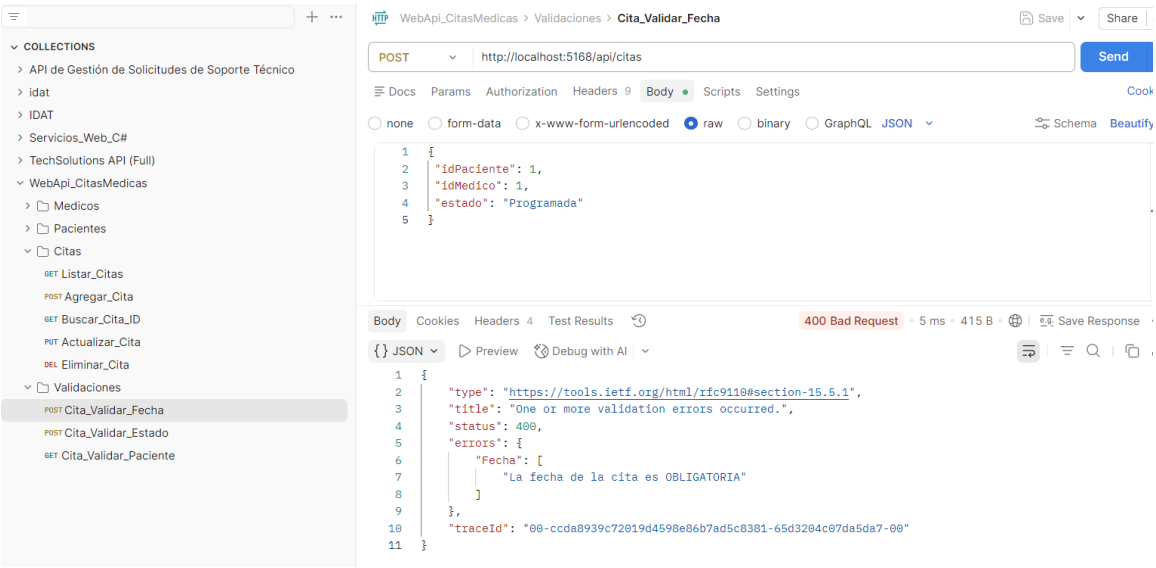


- Notificación en consola por **WhatsApp** de la cancelación de la cita.

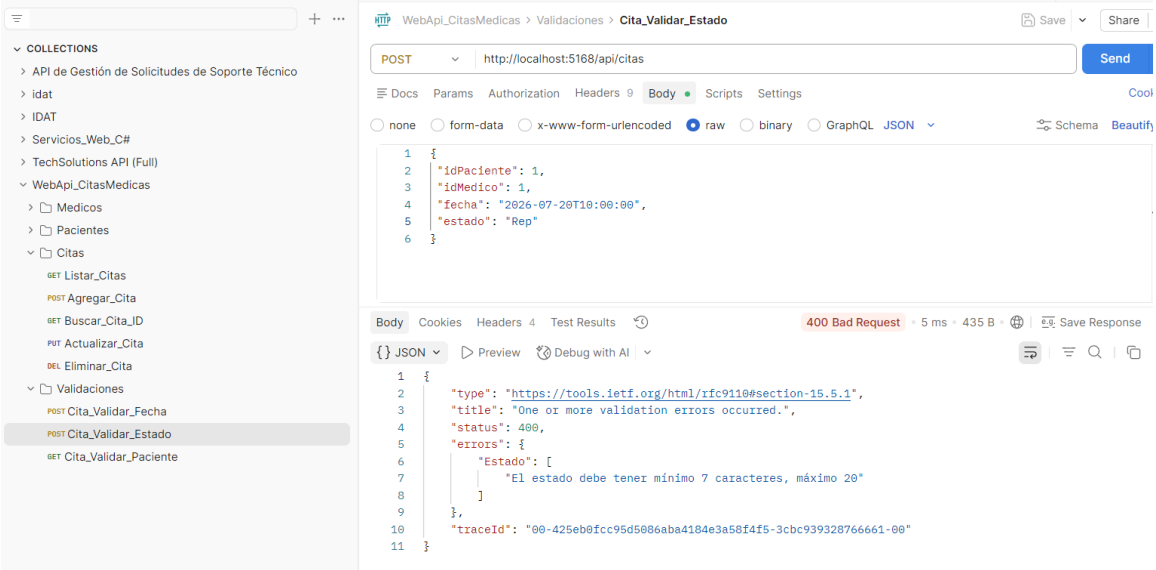
```
PROBLEMAS    SALIDA    CONSOLA DE DEPURACIÓN    TERMINAL    PUERTOS                dotnet
PS D:\Servicios_Web\parcial_grupo06> dotnet run
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\Servicios_Web\parcial_grupo06
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]
      Failed to determine the https port for redirect.
Correo enviado: Su cita médica ha sido registrada
SMS enviado: Su cita médica fue actualizada
WhatsApp enviado: Su cita médica fue cancelada
```

13.6. Validaciones POST Cita (Required / StringLength)

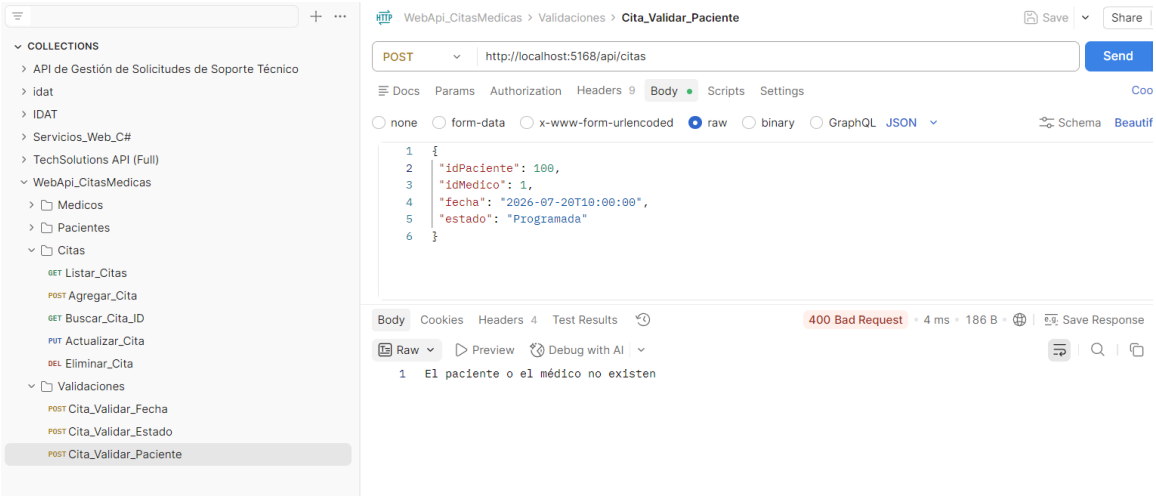
- **Cita_Validar_Fecha:** En esta prueba validamos que, si no enviamos la fecha o le colocamos null, nos indica que este dato es obligatorio.



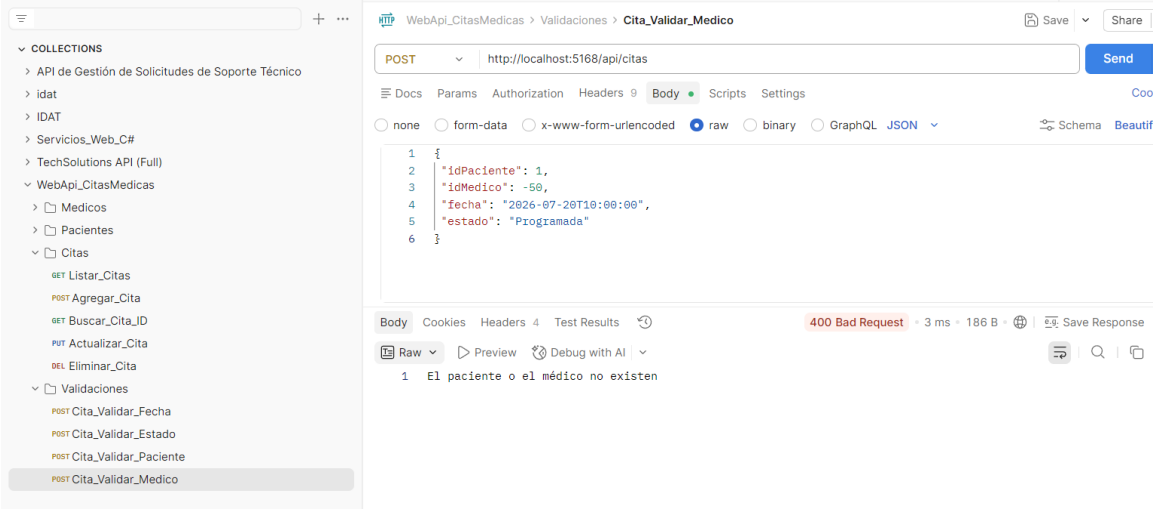
- **Cita_Validar_Estado:** En esta prueba validamos que, si enviamos el estado con una cadena menor a 7 caracteres, nos indicará que este dato es obligatorio.



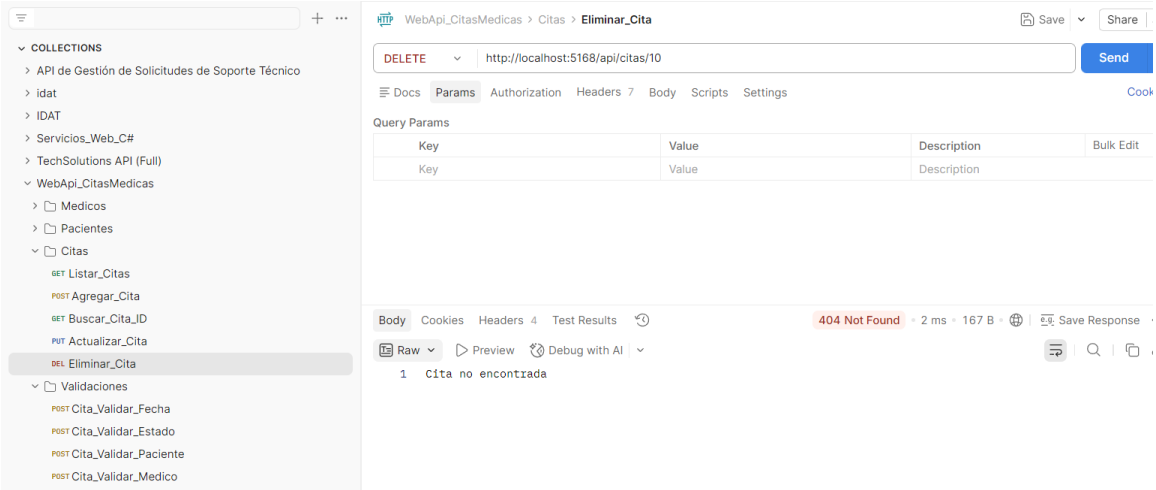
- **Cita_Validar_Paciente:** En esta prueba validamos que, si enviamos un ID de un paciente que no está registrado en sistema, o no mandamos el ID, el endpoint nos devolverá que ese paciente no existe.



- **Cita_Validar_Medico:** En esta prueba validamos que, si enviamos un ID de un médico que no está registrado en sistema, o no mandamos el ID, el endpoint nos devolverá que ese médico no existe.

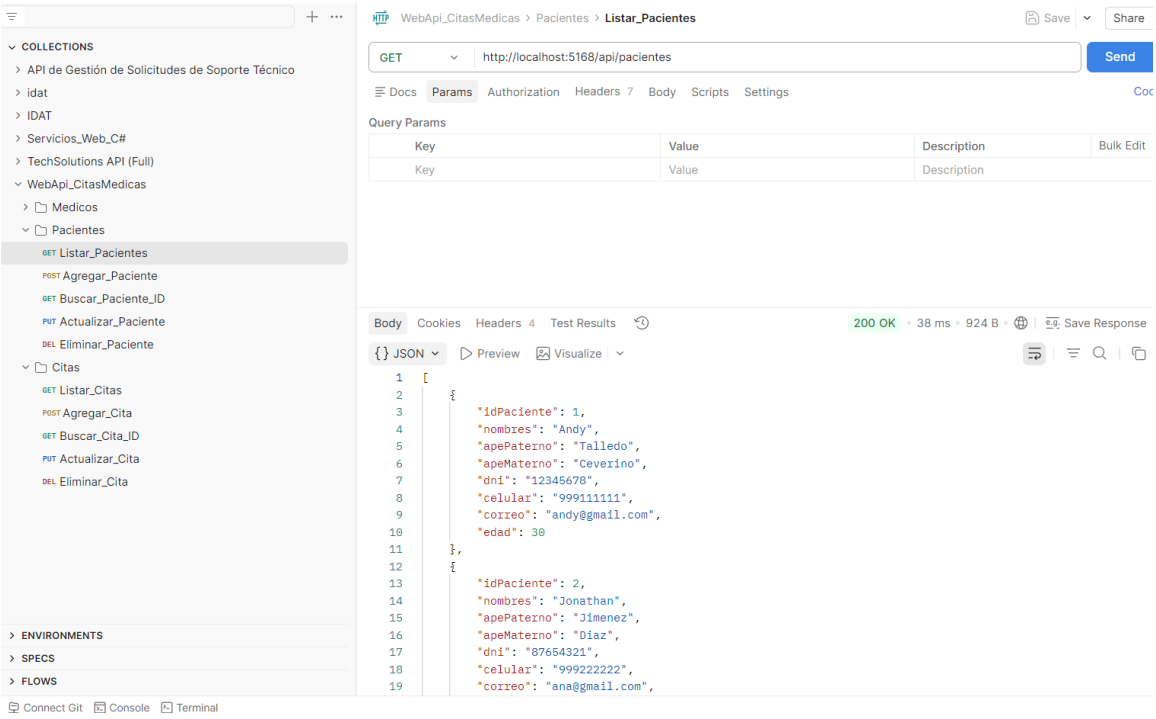


- **Eliminar_Cita:** En esta prueba validamos que, si intentamos eliminar una cita que no existe, el endpoint nos devolverá que no la encuentra.



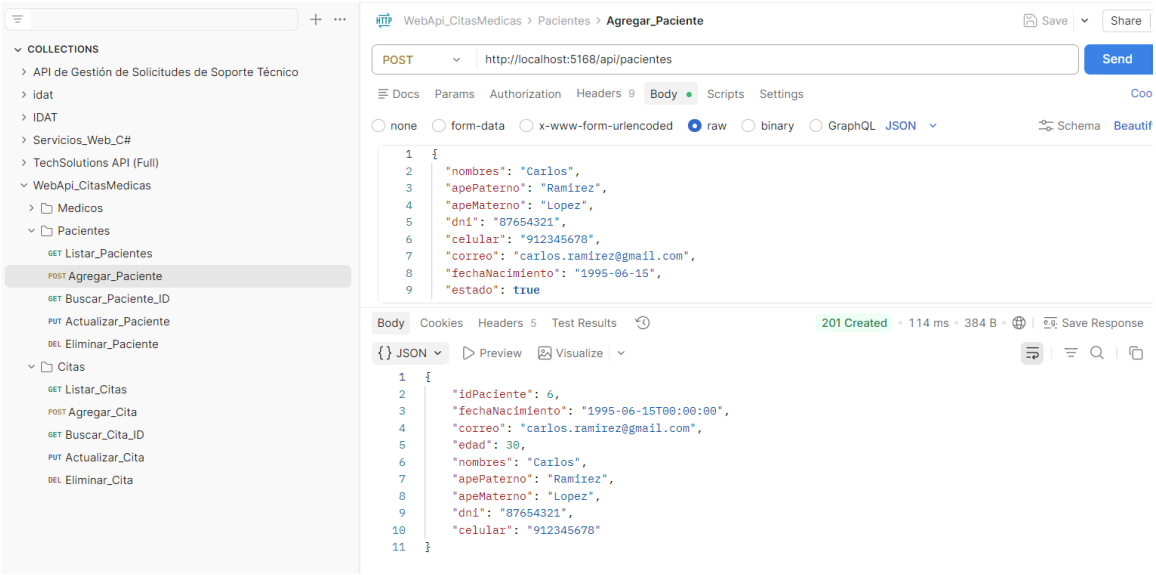
13.7. Prueba GET Pacientes /api/pacientes

Listar_Pacientes: En esta prueba se verifica que el endpoint devuelve correctamente la lista de pacientes registrados en el sistema.



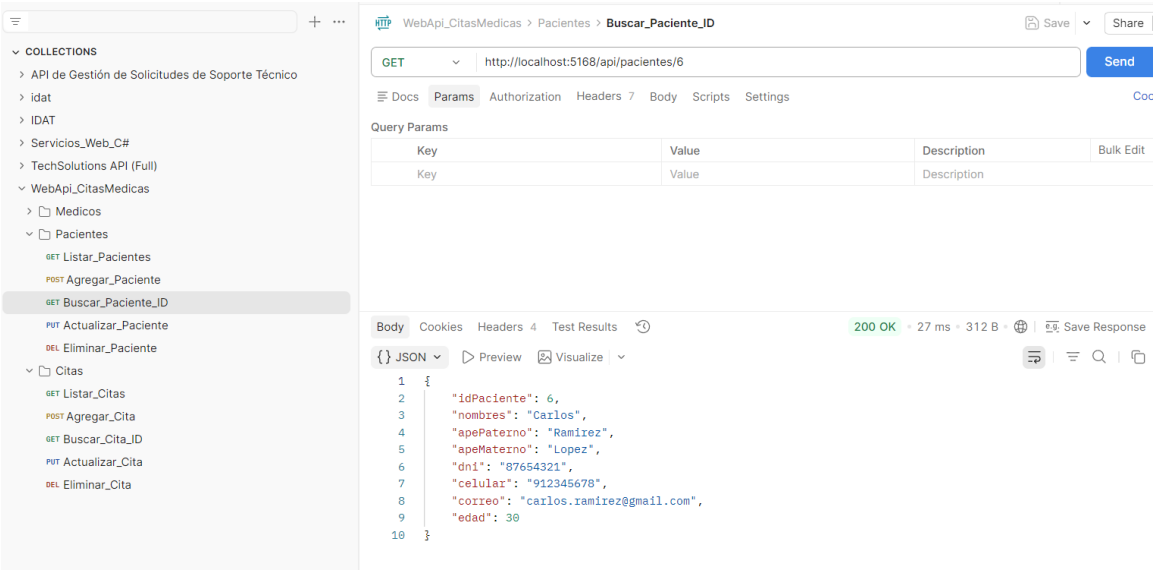
13.8. Prueba POST registro de Paciente /api/pacientes

Agregar_Paciente: En esta prueba se verifica que el endpoint devuelve correctamente 201 Created, agregando el paciente a la lista de pacientes precargados (5).



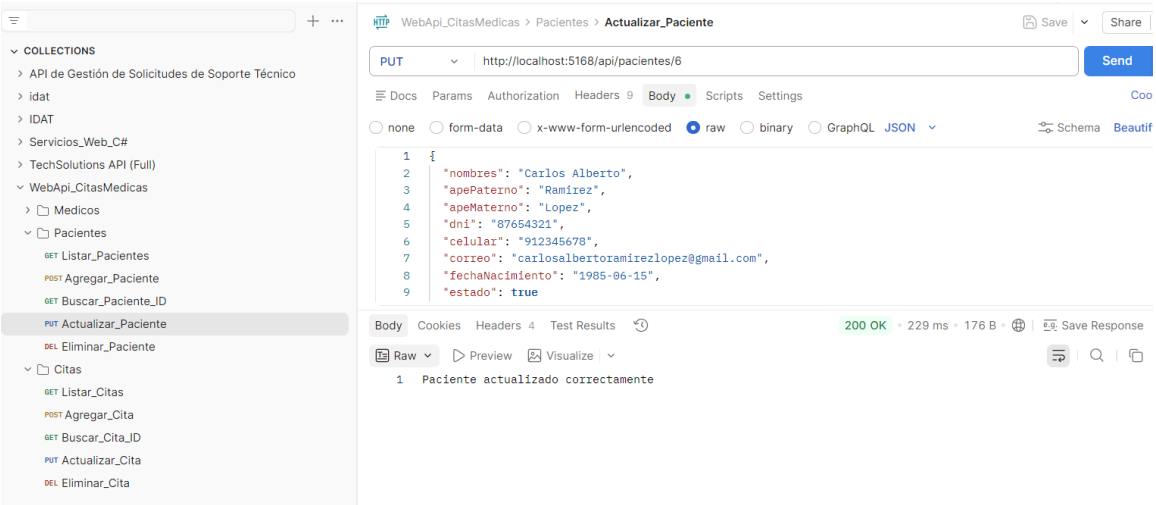
13.9. Prueba GET búsqueda por ID Paciente /api/pacientes/6

Buscar_Paciente_ID: En esta prueba se verifica que el endpoint devuelve correctamente el paciente buscado de acuerdo con su ID.

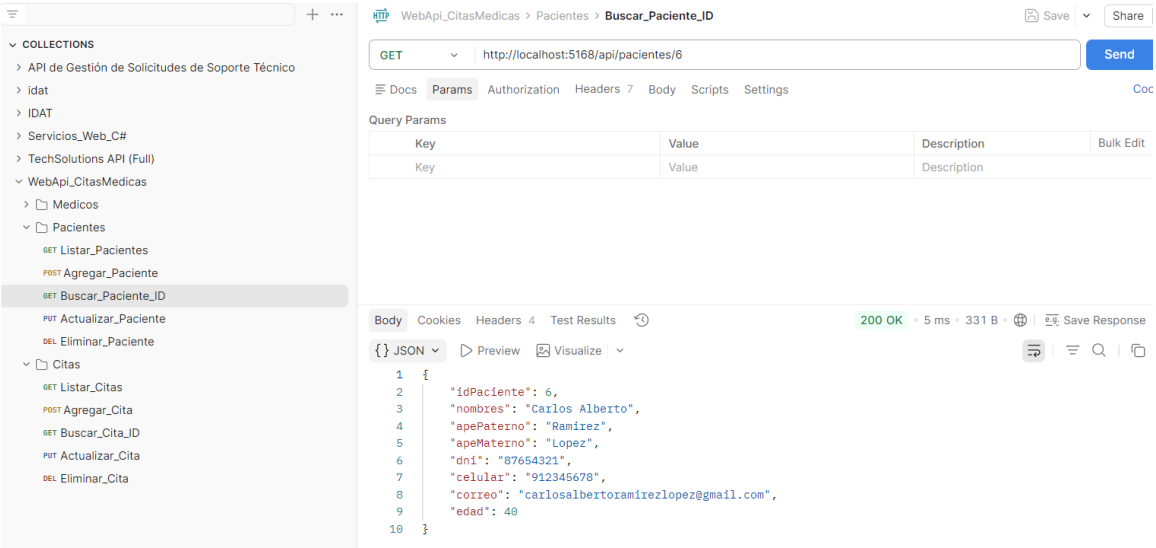


13.10. Prueba PUT actualizar Paciente /api/pacientes/6

Actualizar_Paciente: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, actualizando la información del paciente con ID 6.

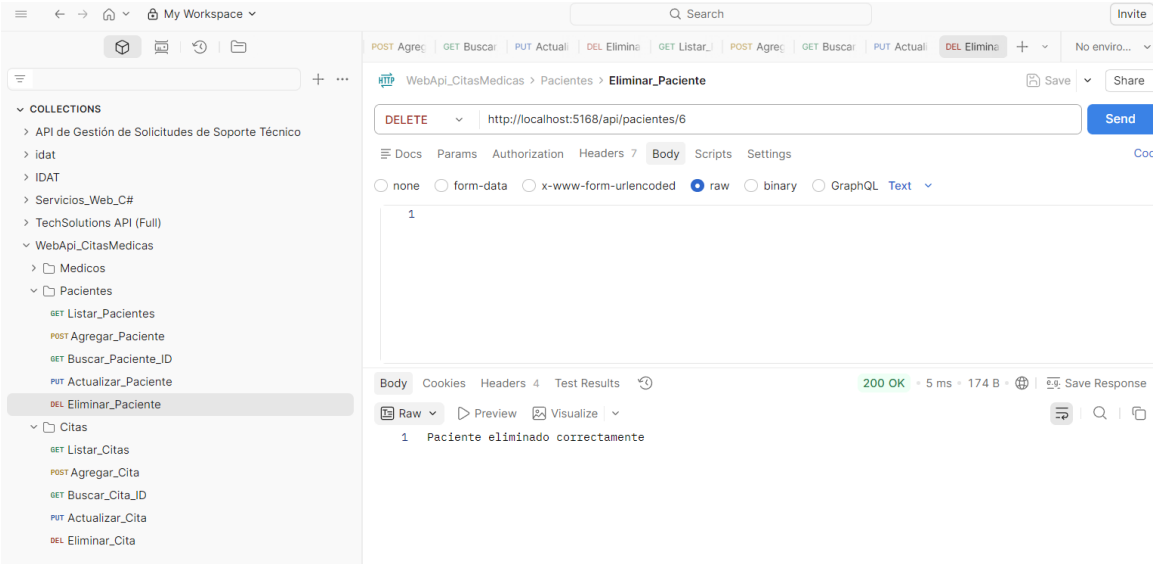


- Correo cambió de carlos.ramirez@gmail.com carlosalbertoramirezlopez@gmail.com
- Fecha de nacimiento cambió de 1995-06-15 a 1985-06-15 (Actualizando edad de 30 a 40)



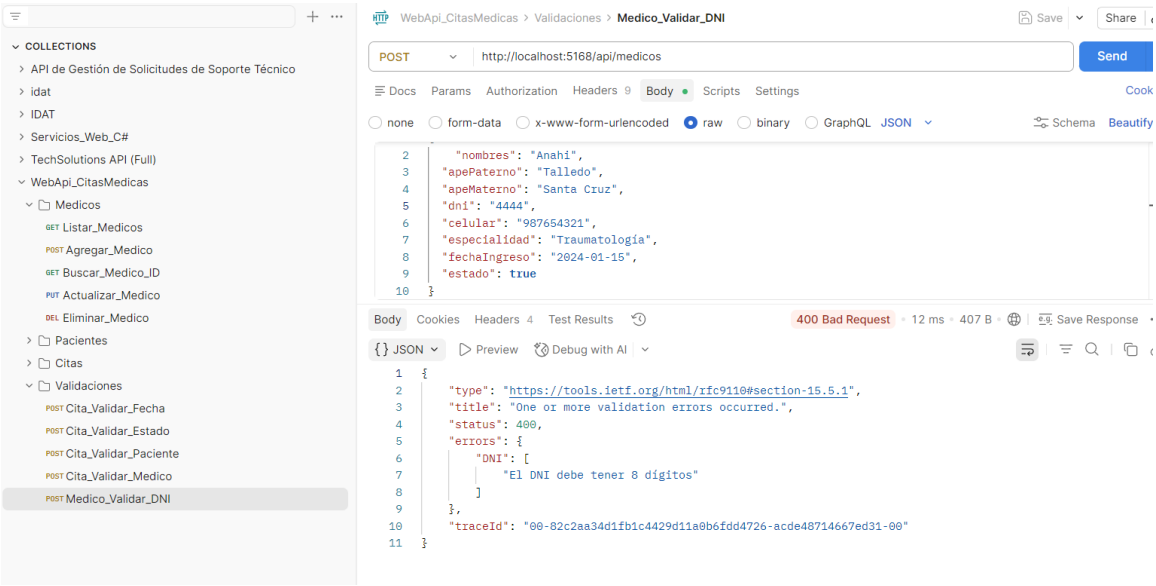
13.11. Prueba DELETE eliminar Paciente /api/pacientes/6

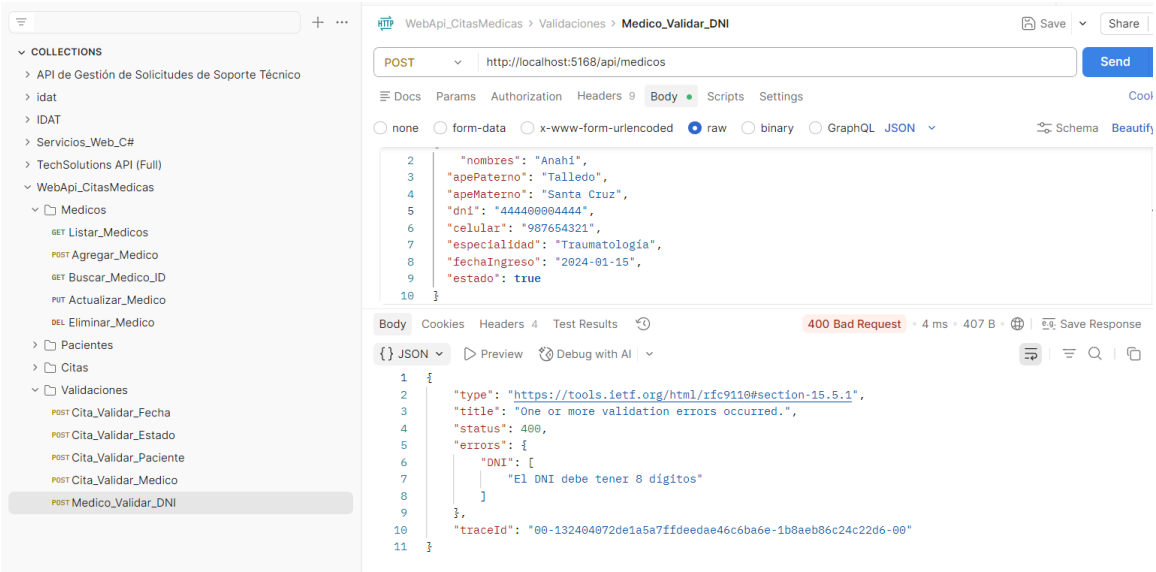
Eliminar_Paciente: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, eliminando el paciente con ID 6.



13.12. Validaciones POST Paciente (Required / StringLength)

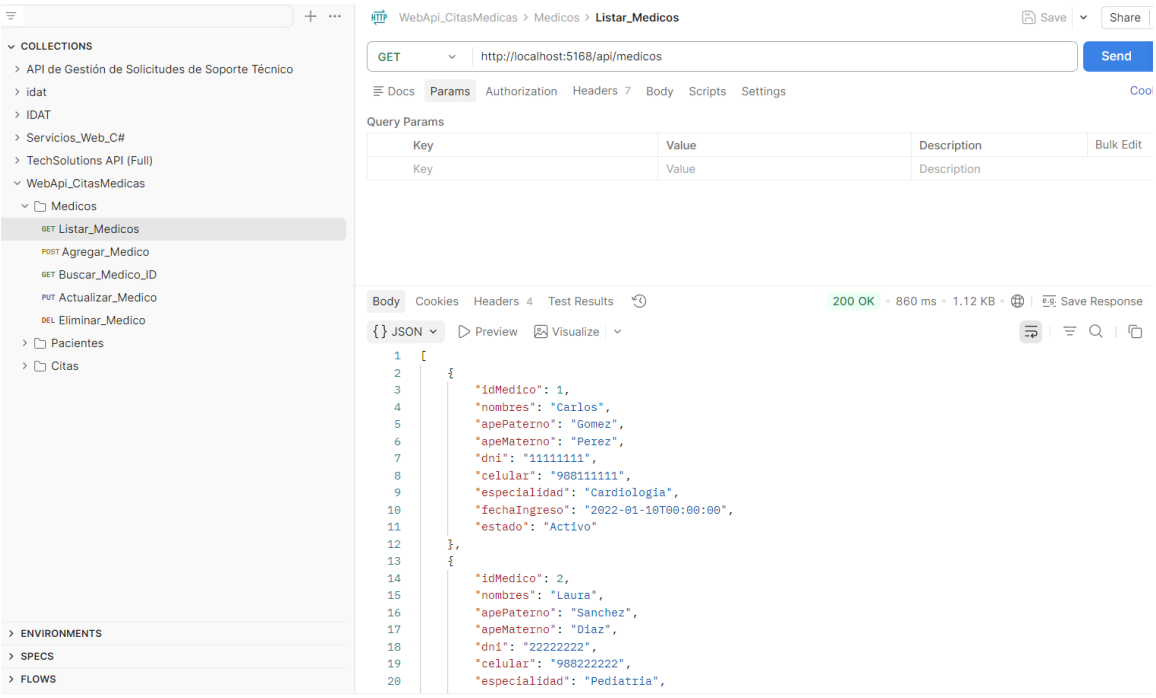
- **Medico_Validar_DNI:** En esta prueba validamos que, si enviamos un DNI que no tenga exactamente 8 dígitos, el endpoint nos devolverá el mensaje informativo correspondiente.





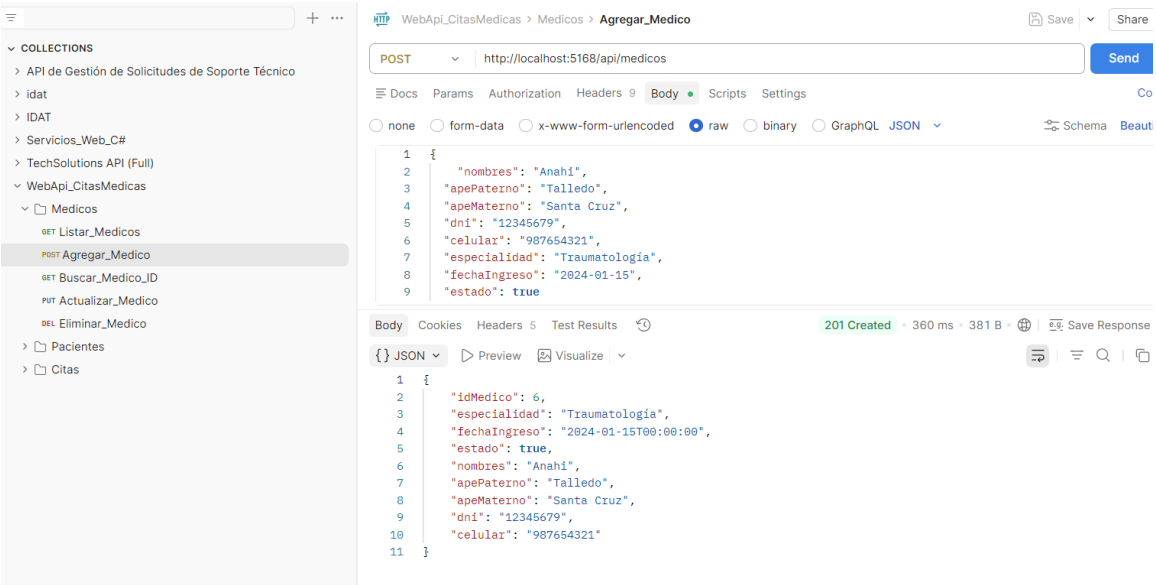
13.13. Prueba GET Médicos /api/medicos

Listar_Medicos: En esta prueba se verifica que el endpoint devuelve correctamente la lista de médicos registrados en el sistema.



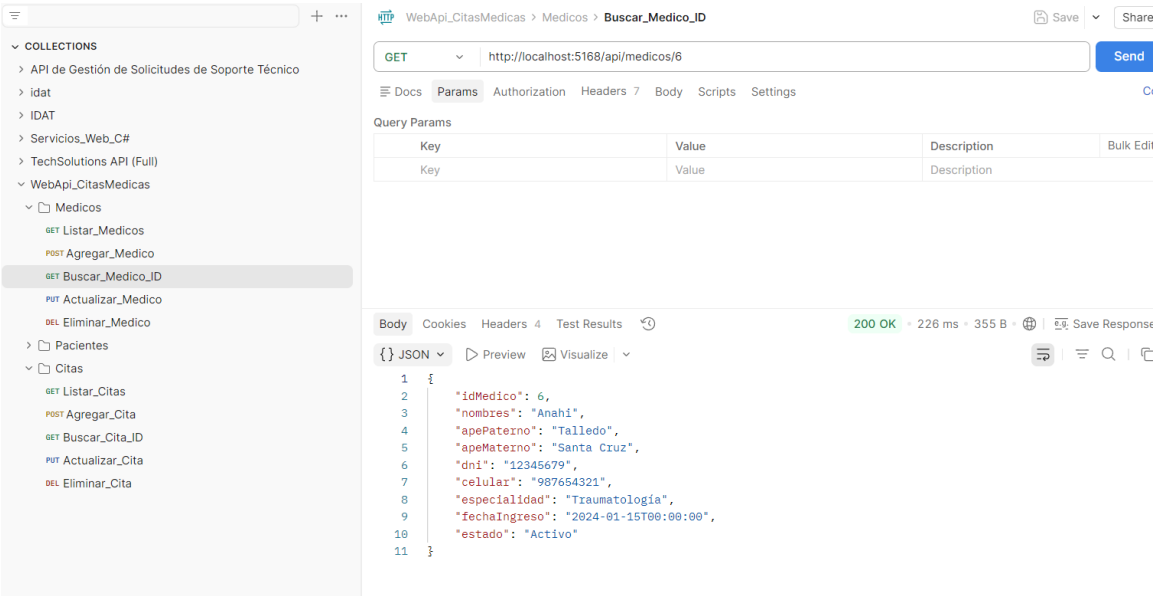
13.14. Prueba POST registro de Médico /api/medicos

Agregar_Medico: En esta prueba se verifica que el endpoint devuelve correctamente 201 Created, agregando el médico a la lista de médicos precargados (5).



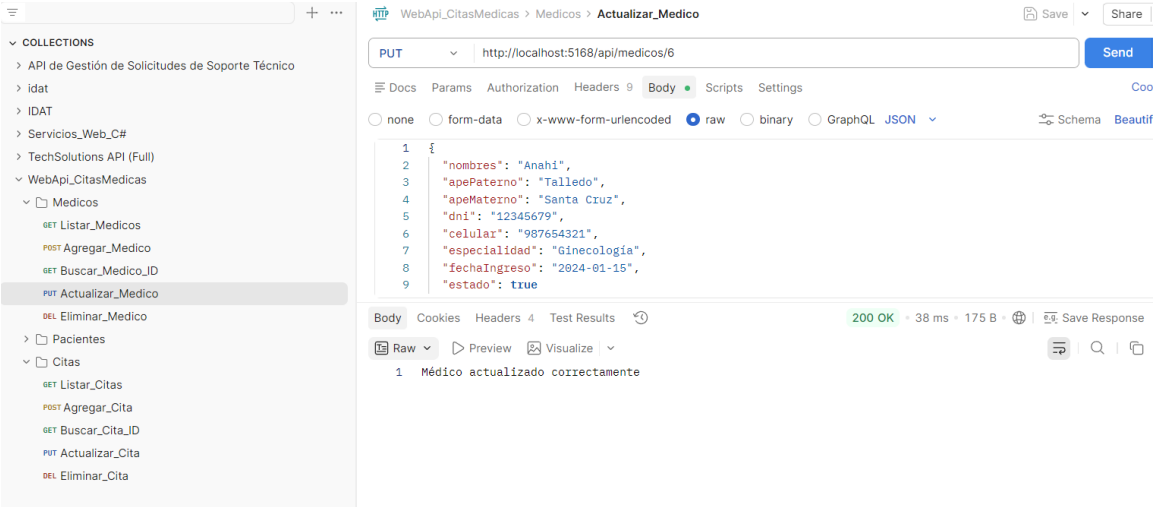
13.15. Prueba GET búsqueda por ID Médicos /api/medicos/6

Buscar_Medico_ID: En esta prueba se verifica que el endpoint devuelve correctamente el médico buscado de acuerdo a su ID.

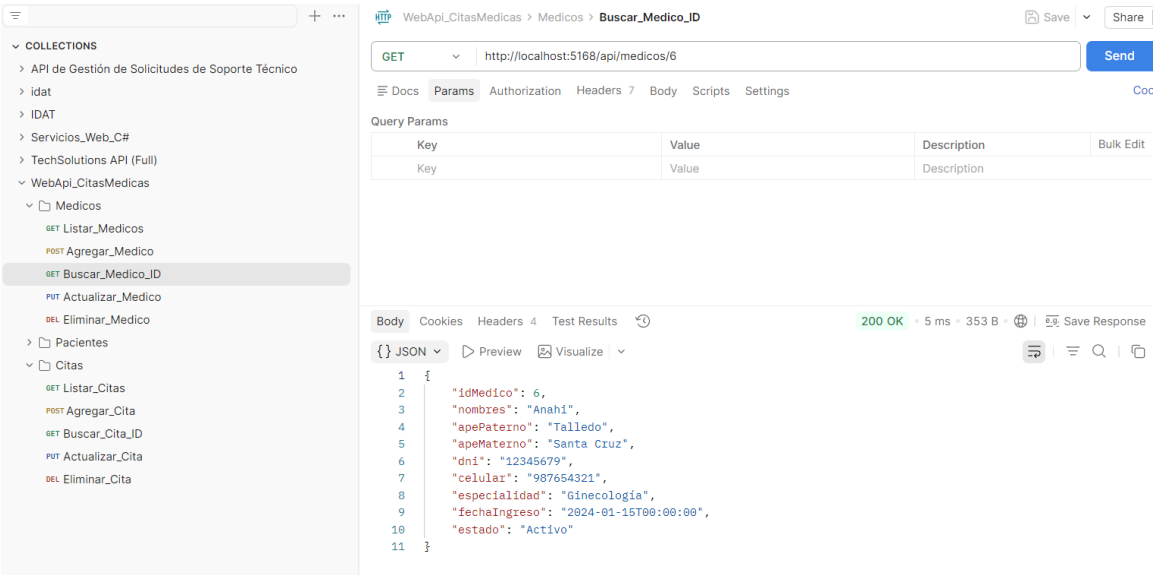


13.16. Prueba PUT actualizar Médico /api/medicos/6

Actualizar_Medico: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, actualizando la información del médico con ID 6.

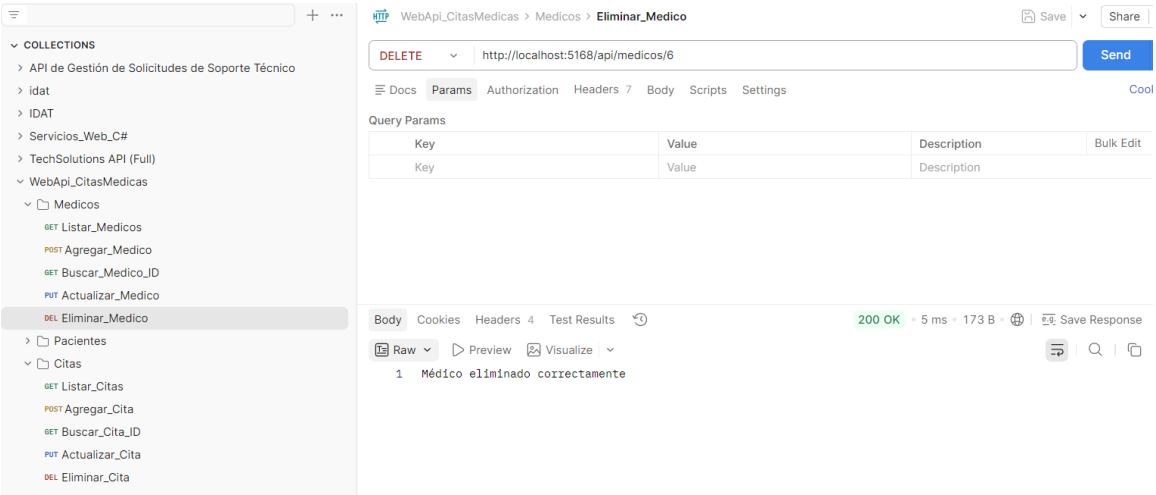


- Especialidad de traumatología a ginecología.



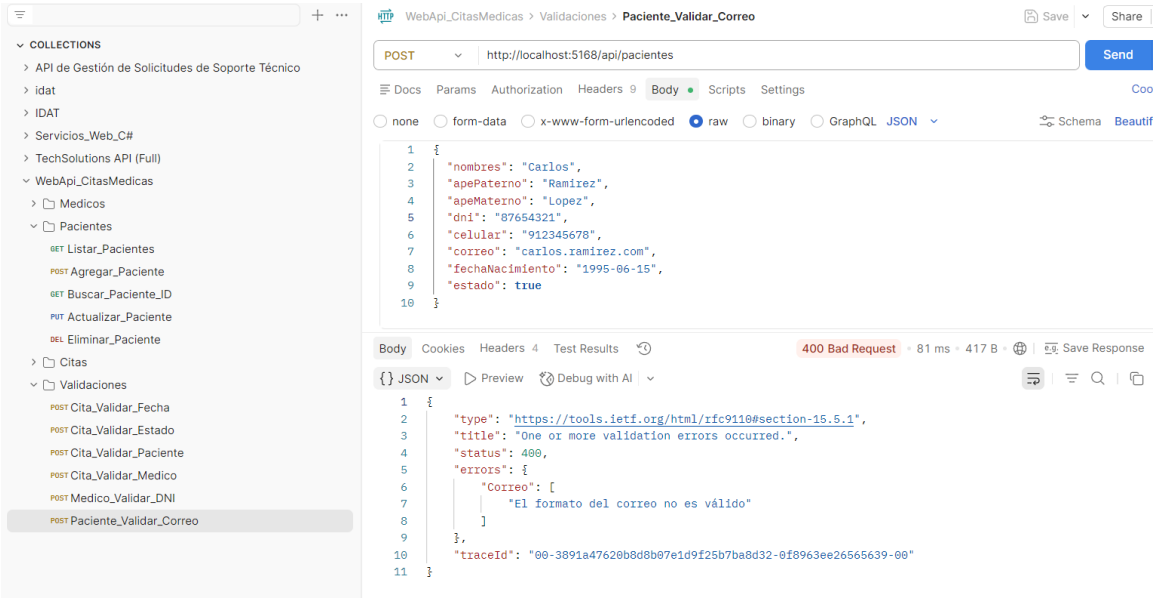
13.17. Prueba DELETE eliminar Médico /api/medicos/6

Eliminar_Medico: En esta prueba se verifica que el endpoint devuelve correctamente 200 OK, eliminando el médico con ID 6.



13.18. Validaciones POST Medico (Required / StringLength)

- **Paciente_Validar_Correo:** En esta prueba validamos que, si enviamos un correo electrónico con un formato no válido, el endpoint nos devolverá el mensaje informativo correspondiente.



14. Conclusiones

El desarrollo de esta API permitió implementar un sistema básico de gestión de citas médicas
utilizando una arquitectura basada en API REST.

Durante el proyecto se aplicaron conceptos importantes como:

- Uso de controladores en ASP.NET Core
- Implementación de endpoints HTTP
- Uso de interfaces para desacoplar funcionalidades
- Validación de datos de entrada
- Pruebas de funcionamiento mediante Postman

El proyecto demuestra los fundamentos necesarios para el desarrollo de APIs modernas.